VoiceOps 2026: ZentroMart AI Support Agent

Akshaynbhat · Capstone report · https://github.com/Akshaynbhat/VoiceOps-Capstone

1. What I built

Pipeline: customer audio → Gnani STT (en-IN, ITN on) → agent (router → controller → retrieval + business rules → LLM reply) → Gnani TTS (timbre-v2.5). The LLM is Groq-hosted (*openai/gpt-oss-20b* for replies, *llama-3.1-8b-instant* as a JSON router). `python run.py <audio_folder> --dataset <dataset>` processes a folder and saves reply text, reply audio and the section IDs used for every file.

Step 1 (prompting): a system prompt setting the role (ZentroMart's AI assistant), limits (no actions, no invented policy, say when unsure) and style. No documents, so it has to guess.

Step 2 (RAG): every `## [SECTION-ID]` heading is one chunk (82 current sections), so the ID travels with the text and is reported back. Hybrid TF-IDF (word 1-2 grams + character 3-5 grams) with a synonym map from customer words to policy words ("send it back" → return window, "nobody came" → reverse pickup), plus an English search query written by the router, which lets Hindi/Hinglish questions find English policies. The superseded returns policy is excluded from search and only fetched when RET-3 says it still applies (electronics delivered before 1 August 2026). Products named in a question are matched to `products.csv` rows and linked to the policy for their return category. The prompt states the precedence rule (policy > agent guide > help > FAQ > marketing) so the FAQ's vague "5 to 7 days" never overrides REF-2.

Deterministic business rules: LLMs get date arithmetic wrong, so business days (Mon-Sat, skipping holidays.csv), refund ETAs (read from the REF-2 table), return deadlines (RET-2/3/8, calendar days), overdue refunds and Plus pickup-fee waivers are computed in Python and handed to the LLM as facts. The numbers are parsed from the KB tables at runtime; nothing is hard-coded per question.

Step 3 (conversation): a small state machine owns the parts that must never go wrong: order IDs parsed even when spoken in groups ("8978 2837", "double four"); verification with phone last-4 *or* pincode, one retry, then no details; tool calls (`lookup_order`, `create_ticket`); ESC-1 handovers (human request, legal threat, unauthorised activity, safety, repeat contact, refund >3 business days overdue, damage > Rs. 10,000). A ticket is mentioned only if `created=True`, and a post-filter removes any sentence claiming an action the code didn't take ("I have cancelled…"). Full chat history and the pending request are kept, so "how long will the refund take?" resolves to the same order and item.

Step 4 (voice): any input format is converted to 16 kHz mono WAV with ffmpeg and split into ≤55 s clips (the REST limit is 60 s). Empty transcripts get "Sorry, I didn't catch that". Replies are written for the ear: 2-3 sentences, dates with month and year, order IDs and RRNs read in groups of four, Rs. read as rupees; Devanagari replies are spoken with hi-IN.

2. Step 1 vs Step 2 results

Scored automatically on all 52 practice questions: a reply passes only if it contains every *must_include* fact (any listed alternative; dates and spoken numbers normalised) and none of the *must_not_include* phrases. Retrieval recall = share of gold section IDs that were retrieved.

	Step 1: prompt only	Step 2: RAG agent
Content accuracy (typed question)	34.6%	94.2%
Content accuracy (asr_text, simulated voice)	n/a	n/a
Decision accuracy (answer/clarify/escalate/refuse)	-	98.1%
Retrieval recall / hit rate	-	100.0% / 100.0%
Content accuracy, language = en	42.4%	97.0%
Content accuracy, language = hi-en	18.2%	90.9%
Content accuracy, language = hi	25.0%	87.5%

Conversations (Step 3): 96 turns in 23 conversations: content 93.8%, decision 100.0%, tool calls 100.0%; 17 conversations fully correct on every turn.

3. What I learned from the comparison

Prompt-only answers sound confident but are usually wrong on anything ZentroMart-specific: return windows per category, the Rs. 49 pickup fee and the Plus waiver, wallet refunds being instant, the festive-sale 30-day window. The model fills gaps with generic e-commerce norms ("5-7 business days", "30-day returns"), which is exactly what the *must_not_include* lists catch. Grounding fixed most of these; the remaining gains came from things retrieval alone doesn't solve: (a) date arithmetic, fixed by computing dates in Python; (b) conflicting documents, fixed by passing doc type/status with each chunk and stating the precedence rule; (c) Hindi questions against English documents, fixed by having the router write an English search query.

4. What went wrong

Step 2 still failed 3 of 52 questions, by intent: warranty (1), unsupported (1), refund_timeline (1). Examples: PQ0005 (warranty: missing 12); PQ0037 (unsupported: missing don't have / do not have / not sure / no information / can't find / not able to find / couldn't find / ■■■■■■■ ■■■■ / pata nahi / jaankari nahi / connect / ■■■■■■); PQ0050 (refund_timeline: missing ZentroMart Wallet / wallet).

Problems hit while building: the first notebook version chunked by fixed length, which split policy tables and lost section IDs; the superseded returns policy leaked old rules (10-day electronics, free pickup) into answers; dates like "17 October 2026" weren't parsed; pandas without `dtype=str` stripped leading zeros from IDs and pincodes; the demo UI hid agent errors behind a bare `except: pass`; and the router sometimes asked for an order ID for questions policy alone can answer ("my order is on its way, can I still stop it?"). Free-tier Groq rate limits made full evaluation runs slow, so the evaluator saves each result and resumes.

5. What I would fix next

• Replace TF-IDF with multilingual embeddings (e.g. a bge-m3-style model) plus a reranker, so Hindi and other Indian languages retrieve directly instead of relying on the router's translation.
• Use Gnani streaming STT/TTS over WebSocket for lower latency in live calls, and barge-in.
• Confidence-based recovery for unclear audio: when the transcript has no order ID where one is expected, or STT confidence is low, read back what was heard and confirm before acting.
• A cancel tool and grocery slot data (CAN-3) so the agent can act, not just explain.
• An LLM-as-judge pass alongside the string-match scorer to catch correct paraphrases it misses.

6. Bonus work

Hindi/Hinglish replies in the customer's script; spoken-number normalisation for ASR output ("eight nine seven eight", "double four"); evaluation on the noisy *asr_text* column to estimate voice robustness; long-audio chunking; offline pytest suite with the LLM stubbed (verification, retry lockout, tickets, business days, retrieval).

All answers are generated by the agent; nothing was typed or edited by hand. Only Gnani is used for speech. Only the synthetic dataset was used.